

Week 10 Homework Report: CSS Grid Layouts

Student Name: Enis Ögdüm **Course:** Dynamic Documents and Front-End Web Development **Date:** 04.12.2025

Overview

This homework assignment involves creating three distinct web layouts using CSS Grid, based on principles of web design. The layouts demonstrate the versatility of CSS Grid for creating responsive, complex, and standard web structures.

Layout 1: Holy Grail Layout

The “Holy Grail” layout is a classic web design pattern consisting of a header, footer, and a main content area flanked by two sidebars (navigation and ads/secondary content).

Implementation Details: - **Grid Structure:** 3 columns (200px - 1fr - 200px) and 3 rows (100px - 1fr - 80px). - **Responsiveness:** Collapses to a single column on smaller screens. - **Usage:** Ideal for blogs, news sites, and documentation.

Core Structure Code

HTML:

```
<!DOCTYPE html>
<html>
  <head>
    <link href="index.css" rel="stylesheet" />
  </head>
  <body>
    <div class="container">
      <div class="header"></div>
      <div class="footer"></div>
      <div class="nav"></div>
      <div class="main"></div>
      <div class="aside"></div>
    </div>
  </body>
</html>
```

CSS:

```
.container {  
  display: grid;  
  grid-template-columns: 200px 1fr 200px;  
  grid-template-rows: 100px 1fr 80px;  
  gap: 10px 10px;  
  grid-auto-flow: row;  
  grid-template-areas:  
    "header header header"  
    "nav main aside"  
    "footer footer footer";  
}  
  
.header { grid-area: header; }  
.footer { grid-area: footer; }  
.nav { grid-area: nav; }  
.main { grid-area: main; }  
.aside { grid-area: aside; }  
  
html, body , .container {  
  height: 100%;  
  margin: 0;  
}
```

Design & Structure

The image displays a web design tool interface, likely LayoutIt!, showing a grid-based layout for a website. The layout is defined by a grid with columns and rows, and areas are labeled: header, footer, nav, main, and aside.

Grid Settings (Left Sidebar):

- Explicit Grid:**
 - grid-template-columns: 200px 1fr 200px
 - grid-template-rows: 100px 1fr 80px
 - row-gap: 10px
 - column-gap: 10px
- Implicit Grid:**
- Grid Placement:**
- Area Box:** width: auto, height: auto
- Children:** .header, .footer

HTML Code (Right Panel):

```
<div class="container">
  <div class="header"></div>
  <div class="footer"></div>
  <div class="nav"></div>
  <div class="main"></div>
  <div class="aside"></div>
</div>
```

CSS Code (Right Panel):

```
.container {
  display: grid;
  grid-template-columns: 200px 1fr 200px;
  grid-template-rows: 100px 1fr 80px;
  gap: 10px 10px;
  grid-template-areas:
    "header header header"
    "nav main aside"
    "footer footer footer";
}
.header { grid-area: header; }
.footer { grid-area: footer; }
.nav { grid-area: nav; }
.main { grid-area: main; }
.aside { grid-area: aside; }
```

Code Editor (Bottom):

The code editor shows the HTML structure of the page, including the DOCTYPE, head, and body tags. The body contains the container div with its sub-elements.

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <link href="index.css" rel="stylesheet" />
5 </head>
6 <body>
7   <div class="container">
8     <div class="header"></div>
9     <div class="footer"></div>
10    <div class="nav"></div>
11    <div class="main"></div>
12    <div class="aside"></div>
13  </div>
14 </body>
15 </html>
16
```

Preview Window (Bottom Right):

The preview window shows the rendered layout of the page, displaying the header, footer, nav, main, and aside areas.

Final Implementation

Wanderlust Blog

Alpine Adventures

Exploring the peaks of the world

Categories

- [Hiking](#)
- [Camping](#)
- [Gear Reviews](#)
- [Photography](#)

Conquering the Summit

The journey to the top was grueling, but the views were worth every step. The air was crisp, and the silence was profound.



We started our ascent at dawn, watching the sun paint the sky in hues of orange and purple. By midday, we had reached the ridge, offering a panoramic view of the valley below.



About Me

Hi, I'm Alex! I travel the world capturing moments.



Newsletter

Subscribe for weekly updates.

Layout 2: Dashboard Layout

A dashboard layout typically requires a more complex grid to organize various widgets, charts, and data points.

Implementation Details: - **Grid Structure:** 4x4 grid with spanning cells. - **Features:** Sidebar for navigation, header for global actions, and various content widgets. - **Usage:** Admin panels, analytics dashboards, and SaaS applications.

Core Structure Code

HTML:

```
<!DOCTYPE html>
<html>
  <head>
    <link href="index.css" rel="stylesheet" />
  </head>
  <body>
    <div class="container">
      <div class="Header"></div>
      <div class="Sidebar"></div>
      <div class="Content 2"></div>
      <div class="Content 3"></div>
      <div class="Footer-"></div>
      <div class="Content 1"></div>
    </div>
  </body>
</html>
```

CSS:

```
.container {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr 1fr;
  grid-template-rows: 100px 200px 200px 100px;
  gap: 15px 15px;
  grid-auto-flow: row;
  grid-template-areas:
    "Header Header Header Header"
    "Sidebar Content 1 Content 1 Content 2"
    "Sidebar Content 3 Content 3 Content 3"
    "Footer- Footer- Footer- Footer-";
}

.Header { grid-area: Header; }
.Sidebar { grid-area: Sidebar; }
.Content\ 2 { grid-area: Content 2; }
.Content\ 3 { grid-area: Content 3; }
.Footer- { grid-area: Footer-; }
.Content\ 1 { grid-area: Content 1; }

html, body , .container {
  height: 100%;
  margin: 0;
}
```

Design & Structure

The screenshot displays the Layoutit! v2 interface, which is a web-based tool for creating and visualizing CSS Grid layouts. The main workspace shows a grid layout with the following components:

- Header:** A blue box at the top.
- Sidebar:** A green box on the left side.
- Content 1-:** A blue box in the top right area.
- Content 2:** A red box in the top right area.
- Content 3:** An orange box in the middle right area.
- Footer-:** A green box at the bottom.

The left sidebar contains configuration options:

- Explicit Grid:** Allows defining grid-template-columns and grid-template-rows with specific values and units.
- Implicit Grid:** Allows defining grid-template-columns and grid-template-rows with specific values and units.
- Grid Placement:** Allows defining grid-template-columns and grid-template-rows with specific values and units.
- Area Box:** Allows defining width and height for the grid area.
- Children:** Allows defining the children of the grid area.

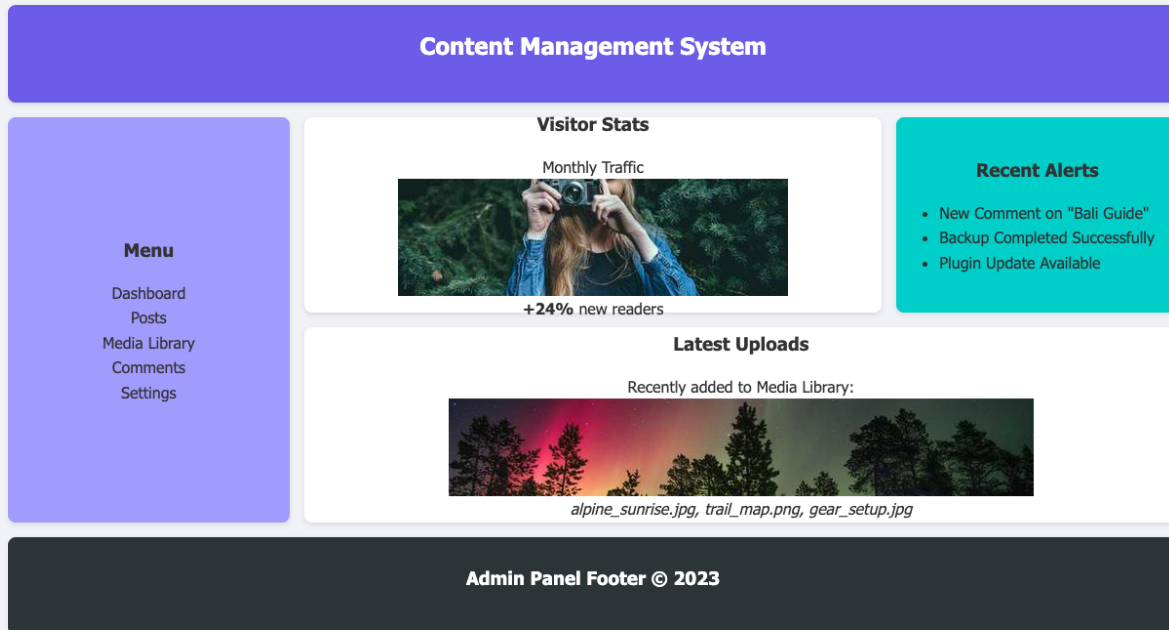
The right sidebar shows the generated HTML and CSS code:

```
HTML
<div class="container">
  <div class="Header"></div>
  <div class="Sidebar"></div>
  <div class="Content 1-"></div>
  <div class="Content 2"></div>
  <div class="Content 3"></div>
  <div class="Footer-"></div>
</div>

CSS
.container {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr 1fr;
  grid-template-rows: 100px 200px 200px 100px;
  gap: 15px 15px;
  grid-template-areas:
    "Header Header Header Header"
    "Sidebar Content 1- Content 1- Content 2"
    "Sidebar Content 3 Content 3 Content 3"
    "Footer- Footer- Footer- Footer-";
}
.Header { grid-area: Header; }
.Sidebar { grid-area: Sidebar; }
.Content 1- { grid-area: Content 1-; }
.Content 2 { grid-area: Content 2; }
.Content 3 { grid-area: Content 3; }
.Footer- { grid-area: Footer-; }
```

Final Implementation

Travel Admin Dashboard



Layout 3: Masonry Gallery

A masonry layout is popular for image galleries where items have varying aspect ratios.

Implementation Details:

- **Grid Structure:** Uses grid-template-columns: repeat(auto-fill, minmax(250px, 1fr)) for responsiveness.
- **Features:** grid-auto-flow: dense allows smaller items to fill gaps left by larger ones.
- **Usage:** Portfolios, photo galleries, and Pinterest-style feeds.

Core Structure Code

HTML:

```
<!DOCTYPE html>
<html>
  <head>
    <link href="index.css" rel="stylesheet" />
  </head>
  <body>
    <div class="container">
      <div class="three"></div>
      <div class="one"></div>
      <div class="two"></div>
      <div class="four"></div>
      <div class="five"></div>
      <div class="six"></div>
      <div class="seven"></div>
      <div class="eight"></div>
      <div class="night"></div>
      <div class="ten"></div>
    </div>
  </body>
</html>
```

CSS:

```
.container {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr 1fr;
  grid-template-rows: 200px 200px 200px 200px;
  gap: 15px 15px;
  grid-auto-flow: row;
  grid-template-areas:
    "one four six seven"
    "two four eight eight"
    "three four eight eight"
    "three five night ten";
}

.three { grid-area: three; }
.one { grid-area: one; }
.two { grid-area: two; }
.four { grid-area: four; }
.five { grid-area: five; }
.six { grid-area: six; }
.seven { grid-area: seven; }
.eight { grid-area: eight; }
.night { grid-area: night; }
.ten { grid-area: ten; }

html, body, .container {
  height: 100%;
  margin: 0;
}
```


Design & Structure

The image displays a web development environment with a grid layout design and its implementation. The top section shows a visual grid layout with colored boxes representing different grid areas. The bottom section shows the corresponding HTML and CSS code.

Grid Layout Design (Top):

- Explicit Grid:** grid-template-columns: 1fr 1fr 1fr 1fr; grid-template-rows: 200px 200px 200px 200px; row-gap: 15px; column-gap: 15px.
- Implicit Grid:** Grid Placement, Area Box, Children.
- Grid Areas:** one, two, three, four, five, six, seven, eight, nine, ten.

HTML Code (Bottom Left):

```
<!DOCTYPE html>
<html>
<head>
<link>
</head>
<body>
<div class="container">
<div class="three"></div>
<div class="one"></div>
<div class="two"></div>
<div class="four"></div>
<div class="five"></div>
<div class="six"></div>
<div class="seven"></div>
<div class="eight"></div>
<div class="nine"></div>
<div class="ten"></div>
</div>
</body>
</html>
```

CSS Code (Bottom Right):

```
.container {
display: grid;
grid-template-columns: 1fr 1fr 1fr 1fr;
grid-template-rows: 200px 200px 200px 200px;
gap: 15px 15px;
grid-template-areas:
"one four six seven"
"two four eight eight"
"three four eight eight"
"three five nine ten";
}
.three { grid-area: three; }
.one { grid-area: one; }
.two { grid-area: two; }
.four { grid-area: four; }
.five { grid-area: five; }
.six { grid-area: six; }
.seven { grid-area: seven; }
.eight { grid-area: eight; }
```

Final Implementation

Travel Photography Portfolio



Tools Used

- **CSS Grid:** For main layout structure.
- **LayoutIt:** Used to prototype the grid areas.
- **Visual Studio Code:** For development.